

Практичне завдання: Рефакторинг системи сповіщень згідно з SOLID

Мета завдання:

Виявити порушення принципів SOLID у наданому коді та виправити їх шляхом рефакторингу.

Умова завдання:

Вам надано початкову реалізацію класу, який відправляє сповіщення користувачам. Код працює, але порушує принципи SOLID.

Ваше завдання:

Виявити, які саме принципи порушено (SRP, OCP, LSP, ISP, DIP); ➤ Провести рефакторинг коду відповідно до кожного з принципів; ➤ Надати фінальний варіант з поясненням, як кожен принцип SOLID реалізовано у вашій версії.

Початковий код (із порушеннями SOLID):

```
public class NotificationService
{
    public void SendNotification(string type, string message) {
        if (type == "email")
        {
            Console.WriteLine($"Sending EMAIL: {message}");
        }
        else if (type == "sms")
        {
            Console.WriteLine($"Sending SMS: {message}");
        }
        else if (type == "push")
        {
            Console.WriteLine($"Sending PUSH: {message}");
        }
    }

    public void SaveNotificationToDatabase(string message)
    {
        Console.WriteLine($"Saving to DB: {message}");
    }

    public void LogNotification(string message)
    {
        Console.WriteLine($"Logging: {message}");
    }
}
```

Завдання для студента:

1. **SRP**: Винести логіку логування, збереження у БД та відправки

повідомлень у окремі класи.

2. **OCP**: Зробити так, щоб можна було додати новий тип сповіщення (наприклад, Telegram) без модифікації NotificationService.
3. **LSP**: Забезпечити, щоб нові типи сповіщень могли повноцінно замінити інші без порушення логіки.
4. **ISP**: Якщо створюється інтерфейс з багатьма методами, розділіть його на більшу кількість, але з більш вузьким колом можливостей.
5. **DIP**: Замініть залежність від конкретної логіки відправки на залежність від абстракцій (через інтерфейси).

В результаті ви отримаєте новий код, в якому в коментарях вкажіть де і як застосовано принципи **SOLID**.